SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

Schema Used in All Questions:

Student (RollNo, Name, DeptID, Age, Marks) | Department (DeptID, DeptName, HOD)

Sample Data (load this into MySQL before attempting the questions):

Department

DeptID	DeptName	HOD
1	Computer Science	Dr. Rao
2	Electronics	Dr. Iyer
3	Mechanical	Dr. Nair

Student

RollNo	Name	DeptID	Age	Marks
101	Aravind	1	20	85
102	Bhavya	1	21	78
103	Chandran	1	20	90
104	Deepak	2	22	60
105	Esha	2	21	55
106	Farhan	2	22	95
107	Gowri	3	20	40
108	Hari	3	21	45
109	Indra	3	20	68

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**

1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.
	Given Query:
	`SELECT s.Name, d.DeptName`
	`FROM Student s, Department d;`
	`-- Intent: list each student with their own department's name`
		BL3 – Apply	**5**

2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.
	Given Query:
	`SELECT s.Name, s.DeptID, s.Marks`
	`FROM Student s`
	`WHERE s.Marks > (SELECT AVG(Marks) FROM Student);`
	`-- Intent: list students who score above their OWN department's average`
		BL3 – Apply	**5**

3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.
	Given Query:
	`CREATE VIEW DeptAvgMarks AS`
	`SELECT Name, DeptID, AVG(Marks) AS AvgMarks`
	`FROM Student`
	`GROUP BY DeptID;`
	
	`SELECT * FROM DeptAvgMarks;`
		BL3 – Apply	**5**

4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.
	Given Query:
	`π(Name, Age) (Student) ⋈ Department`
	`-- Intent: natural join of Student and Department on DeptID`
		BL3 – Apply	**5**

5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.
	Given Query:
	`CREATE TABLE Student (`
	`  RollNo INT PRIMARY KEY,`
	`  Name  VARCHAR(50),`
	`  DeptID INT,`
	`  Age  INT,`
	`  Marks  INT`
	`);`
	`-- Department table only has DeptID values 1, 2, 3`
	
	`INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)`
	`VALUES (110, 'Arun', 9, 20, 78);`
		BL3 – Apply	**5**

6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.
	Given Query:
	`SELECT s.Name,`
	`(SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID) AS DeptName`
	`FROM Student s;`
		BL6 – Create	**5**

7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.
	Given Query:
	`SELECT DeptID, AVG(Marks) AS AvgMarks`
	`FROM Student`
	`GROUP BY RollNo;`
		BL3 – Apply	**5**

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.
	Given Query:
	`-- Intent: give 5 bonus marks to students in Department 2 only`
	`UPDATE Student`
	`SET Marks = Marks + 5;`
		BL3 – Apply	**5**

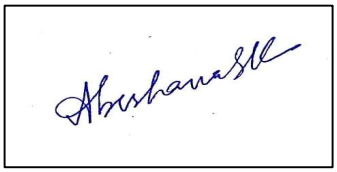
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINs and using appropriate indexes.
	Given Query:
	`SELECT s.Name, s.Marks,`
	`(SELECT DeptName FROM Department WHERE DeptID = s.DeptID) AS DeptName,`
	`(SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS HeadOfDept`
	`FROM Student s`
	`WHERE s.DeptID IN (`
	`SELECT DeptID FROM Department WHERE DeptName IN ('Computer Science', 'Electronics')`
	`);`
		BL6 – Create	**5**

10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.
	Given Query:
	`-- Intent: students who scored more than AT LEAST ONE student in Department 2`
	`{ s.Name | Student(s) ∧ (∀ t) (Student(t) ∧ t.DeptID = 2 → s.Marks > t.Marks) }`
		BL3 – Apply	**5**

Code of Conduct:

I Abishana S K(192511242) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1) Given Query

```
SELECT s.Name, d.DeptName
FROM Student s, Department d;
```

Error:

The query does not specify a join condition between the Student and Department tables. Therefore, it returns a Cartesian Product, where every student is paired with every department.

Correct Query:

```
SELECT s.Name, d.DeptName
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID;
```

Explanation:

The INNER JOIN matches each student with the corresponding department using the common attribute DeptID. This eliminates duplicate combinations and returns only the required records.

Optimization:

Using INNER JOIN with the join condition reduces unnecessary row combinations and improves query efficiency by retrieving only the matching records.

Run the given query:

```
SELECT s.Name, d.DeptName
FROM Student s, Department d;
```

```
mysql> SELECT s.Name, d.DeptName
    -> FROM Student s, Department d;
+----------+------------------+
| Name     | DeptName         |
+----------+------------------+
| Aravind  | Mechanical       |
| Aravind  | Electronics      |
| Aravind  | Computer Science |
| Bhavya   | Mechanical       |
| Bhavya   | Electronics      |
| Bhavya   | Computer Science |
| Chandran | Mechanical       |
| Chandran | Electronics      |
| Chandran | Computer Science |
| Deepak   | Mechanical       |
| Deepak   | Electronics      |
| Deepak   | Computer Science |
| Esha     | Mechanical       |
| Esha     | Electronics      |
| Esha     | Computer Science |
| Farhan   | Mechanical       |
| Farhan   | Electronics      |
| Farhan   | Computer Science |
| Gowri    | Mechanical       |
| Gowri    | Electronics      |
| Gowri    | Computer Science |
| Hari     | Mechanical       |
| Hari     | Electronics      |
| Hari     | Computer Science |
| Indra    | Mechanical       |
| Indra    | Electronics      |
| Indra    | Computer Science |
+----------+------------------+
27 rows in set (0.01 sec)
```

Run the corrected query:

SELECT s.Name, d.DeptName
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID;

```
mysql> SELECT s.Name, d.DeptName
    -> FROM Student s
    -> INNER JOIN Department d
    -> ON s.DeptID = d.DeptID;
+----------+------------------+
| Name     | DeptName         |
+----------+------------------+
| Aravind  | Computer Science |
| Bhavya   | Computer Science |
| Chandran | Computer Science |
| Deepak   | Electronics      |
| Esha     | Electronics      |
| Farhan   | Electronics      |
| Gowri    | Mechanical       |
| Hari     | Mechanical       |
| Indra    | Mechanical       |
+----------+------------------+
9 rows in set (0.01 sec)
```

2) Given Query

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks > (SELECT AVG(Marks) FROM Student);
```

Error:

The subquery calculates the average marks of all students instead of the average marks of the student's own department.

Correct Query:

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks > (
   SELECT AVG(s2.Marks)
   FROM Student s2
   WHERE s2.DeptID = s.DeptID
);
```

Explanation:

The subquery is correlated using DeptID, so it calculates the average marks for the same department as the current student. Only students scoring above their department average are displayed.

Optimization:
A correlated subquery ensures accurate department-wise comparison instead of comparing with the overall average.

MySQL Execution
Run the given query:

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks > (SELECT AVG(Marks) FROM Student);
```

Incorrect output:

```
mysql> SELECT s.Name, s.DeptID, s.Marks
    -> FROM Student s
    -> WHERE s.Marks > (SELECT AVG(Marks) FROM Student);
+----------+--------+-------+
| Name     | DeptID | Marks |
+----------+--------+-------+
| Aravind  |      1 |    85 |
| Bhavya   |      1 |    78 |
| Chandran |      1 |    90 |
| Farhan   |      2 |    95 |
+----------+--------+-------+
4 rows in set (0.01 sec)
```

Run the corrected query:

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks > (
   SELECT AVG(s2.Marks)
   FROM Student s2
   WHERE s2.DeptID = s.DeptID
);
```

Correct Output:

```
mysql> SELECT s.Name, s.DeptID, s.Marks
    -> FROM Student s
    -> WHERE s.Marks > (
    ->     SELECT AVG(s2.Marks)
    ->     FROM Student s2
    ->     WHERE s2.DeptID = s.DeptID
    -> );
+----------+--------+-------+
| Name     | DeptID | Marks |
+----------+--------+-------+
| Aravind  |      1 |    85 |
| Chandran |      1 |    90 |
| Farhan   |      2 |    95 |
| Indra    |      3 |    68 |
+----------+--------+-------+
4 rows in set (0.00 sec)
```

The corrected query returns **Indra** as well because **68 is above the average marks of the Mechanical department** (40, 45, 68 → average = 51). This matches the intended requirement of finding students who scored above **their own department's average**.

3) Given Query

```
CREATE VIEW DeptAvgMarks AS
SELECT Name, DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;

SELECT * FROM DeptAvgMarks;
```

Error:

The query selects the Name column without including it in the GROUP BY clause or using an aggregate function. This causes an error because each department contains multiple student names.

Correct Query:

```
CREATE VIEW DeptAvgMarks AS
SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
```

To display the view:

```
SELECT * FROM DeptAvgMarks;
```

Explanation:
The corrected view groups students by DeptID and calculates the average marks for each department. Removing the Name column ensures that all selected columns are either grouped or aggregated.
Optimization:
The optimized view contains only the required grouped data, reducing unnecessary columns and ensuring correct aggregation.

MySQL Execution
Run the given query:
CREATE VIEW DeptAvgMarks AS
SELECT Name, DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
Output (Incorrect):

```
ERROR 1050 (42S01): Table 'DeptAvgMarks' already exists
mysql> SELECT * FROM DeptAvgMarks;
ERROR 1055 (42000): Expression #1 of SELECT list is not in GROUP BY clause a
nd contains nonaggregated column 'csa05_db.student.Name' which is not functi
onally dependent on columns in GROUP BY clause; this is incompatible with sq
l_mode=only_full_group_by
mysql>
```

the corrected query:
CREATE VIEW DeptAvgMarks AS
SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;

```
mysql> CREATE VIEW DeptAvgMarks AS
    -> SELECT DeptID, AVG(Marks) AS AvgMarks
    -> FROM Student
    -> GROUP BY DeptID;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT * FROM DeptAvgMarks;
+--------+----------+
| DeptID | AvgMarks |
+--------+----------+
|      1 |  84.3333 |
|      2 |  70.0000 |
|      3 |  51.0000 |
+--------+----------+
3 rows in set (0.01 sec)
```

4) Given Expression

π(Name, Age) (Student) ⋈ Department

Error:

The natural join operation is incorrectly written. The projection (π) is applied before performing the join, causing the common attribute DeptID to be removed. As a result, the natural join cannot be performed correctly.

Correct Expression

π(Name, Age) (Student ⋈ Department)

Explanation:

The Student and Department relations are first joined using the common attribute DeptID. After the join is completed, the required attributes Name and Age are projected from the result.

Optimization:

Performing the join before projection preserves the common attribute (DeptID) required for the natural join and produces the correct output.

Expected Result

Name	Age
Aravind	20
Bhavya	21
Chandran	20
Deepak	22
Esha	21

Farhan	22
Gowri	20
Hari	21
Indra	20

5) Given Query

```
CREATE TABLE Student (
   RollNo INT PRIMARY KEY,
   Name VARCHAR(50),
   DeptID INT,
   Age INT,
   Marks INT
);

-- Department table only has DeptID values 1, 2, 3

INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)
VALUES (110, 'Arun', 9, 20, 78);
```

Error:

The value DeptID = 9 does not exist in the Department table. This violates referential integrity.

Correct Query:

```
CREATE TABLE Student (
   RollNo INT PRIMARY KEY,
   Name VARCHAR(50),
   DeptID INT,
   Age INT,
   Marks INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)
VALUES (110, 'Arun', 2, 20, 78);
```

Explanation:

The FOREIGN KEY constraint ensures that every DeptID in the Student table must already exist in the Department table. Since only DeptID

values 1, 2, and 3 exist, changing the value from 9 to 2 satisfies referential integrity.

Optimization:

Using a foreign key constraint prevents invalid department IDs from being inserted, thereby maintaining data consistency and integrity.

MySQL Execution

If your Student table **already exists** (from the initial setup), **do not create it again**. Instead, demonstrate only the insert statement.

Run the given query:

```
INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)
VALUES (110, 'Arun', 9, 20, 78);
```

Output (Incorrect):

```
mysql> INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)
    -> VALUES (110, 'Arun', 9, 20, 78);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`csa
05_db`.`student`, CONSTRAINT `student_ibfk_1` FOREIGN KEY (`DeptID`) REFERENCES `departmen
t` (`DeptID`))
mysql>
```

Run the corrected query:

```
INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)
VALUES (110, 'Arun', 2, 20, 78);
```

Verify:

```
SELECT * FROM Student
WHERE RollNo = 110;
```

Correct Output:

```
mysql> INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)
    -> VALUES (110, 'Arun', 2, 20, 78);
```

```
mysql> SELECT * FROM Student
    -> WHERE RollNo = 110;
+--------+------+--------+------+-------+
| RollNo | Name | DeptID | Age  | Marks |
+--------+------+--------+------+-------+
|    110 | Arun |      2 |   20 |    78 |
+--------+------+--------+------+-------+
1 row in set (0.00 sec)
```

6) Given Query

```
SELECT s.Name,
      (SELECT d.DeptName
       FROM Department d
```

```
    WHERE d.DeptID = s.DeptID) AS DeptName
FROM Student s;
```

Error:

The query uses a subquery to fetch the department name for each student. This subquery is executed once for every row in the Student table, making it less efficient.

Correct Query:

```
SELECT s.Name, d.DeptName
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID;
```

Explanation:

The INNER JOIN retrieves the student name and department name by joining the Student and Department tables using DeptID. This avoids executing a subquery for every row.

Optimization:

Using INNER JOIN improves performance by retrieving the required data in a single operation instead of repeatedly executing the subquery.

MySQL Execution

Run the given query:

```
SELECT s.Name,
    (SELECT d.DeptName
     FROM Department d
     WHERE d.DeptID = s.DeptID) AS DeptName
FROM Student s;
```

Incorrect output:

```
mysql> SELECT s.Name,
    ->        (SELECT d.DeptName
    ->         FROM Department d
    ->         WHERE d.DeptID = s.DeptID) AS DeptName
    -> FROM Student s;
+----------+------------------+
| Name     | DeptName         |
+----------+------------------+
| Aravind  | Computer Science |
| Bhavya   | Computer Science |
| Chandran | Computer Science |
| Deepak   | Electronics      |
| Esha     | Electronics      |
| Farhan   | Electronics      |
| Gowri    | Mechanical       |
| Hari     | Mechanical       |
| Indra    | Mechanical       |
| Arun     | Electronics      |
+----------+------------------+
10 rows in set (0.00 sec)
```

Run the corrected query:

SELECT s.Name, d.DeptName
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID;

```
mysql> SELECT s.Name, d.DeptName
    -> FROM Student s
    -> INNER JOIN Department d
    -> ON s.DeptID = d.DeptID;
+----------+------------------+
| Name     | DeptName         |
+----------+------------------+
| Aravind  | Computer Science |
| Bhavya   | Computer Science |
| Chandran | Computer Science |
| Deepak   | Electronics      |
| Esha     | Electronics      |
| Farhan   | Electronics      |
| Arun     | Electronics      |
| Gowri    | Mechanical       |
| Hari     | Mechanical       |
| Indra    | Mechanical       |
+----------+------------------+
10 rows in set (0.00 sec)
```

Both queries produce the same output. The JOIN version is preferred because it is more efficient and easier to optimize, especially for larger tables.

7) Given Query:

```
SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY RollNo;
```

Error:

The query groups the records by RollNo instead of DeptID. Since RollNo is unique for each student, each group contains only one record, resulting in incorrect department-wise averages.

Correct Query:

```
SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
```

Explanation:

Grouping by DeptID combines students belonging to the same department, allowing the AVG() function to calculate the average marks for each department correctly.

Optimization:

Using the correct grouping column reduces unnecessary grouping and produces meaningful aggregated results.

MySQL Execution

Run the given query:

```
SELECT DeptID, AVG(Marks) AS AvgMarks
```

FROM Student
GROUP BY RollNo;

Output (Incorrect):

```
mysql> SELECT DeptID, AVG(Marks) AS AvgMarks
    -> FROM Student
    -> GROUP BY RollNo;
+--------+----------+
| DeptID | AvgMarks |
+--------+----------+
|      1 |  85.0000 |
|      1 |  78.0000 |
|      1 |  90.0000 |
|      2 |  60.0000 |
|      2 |  55.0000 |
|      2 |  95.0000 |
|      3 |  40.0000 |
|      3 |  45.0000 |
|      3 |  68.0000 |
|      2 |  78.0000 |
+--------+----------+
10 rows in set (0.02 sec)
```

Run the corrected query:

SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;

Correct Output:

```
mysql> SELECT DeptID, AVG(Marks) AS AvgMarks
    -> FROM Student
    -> GROUP BY DeptID;
+--------+----------+
| DeptID | AvgMarks |
+--------+----------+
|      1 |  84.3333 |
|      2 |  72.0000 |
|      3 |  51.0000 |
+--------+----------+
3 rows in set (0.00 sec)
```

8) Given Query

UPDATE Student
SET Marks = Marks + 5;
Intent: Give 5 bonus marks to students in **Department 2** only.

Error:

The UPDATE statement does not contain a WHERE clause. As a result, it updates the marks of **all students** instead of only those in Department 2.
Correct Query:

UPDATE Student
SET Marks = Marks + 5
WHERE DeptID = 2;
Explanation:

The WHERE clause restricts the update to students belonging to Department 2. This ensures that only the intended records are modified.
Optimization:

Using a WHERE clause prevents unnecessary updates, reduces processing time, and maintains data accuracy.

MySQL Execution
Run the given query:
UPDATE Student
SET Marks = Marks + 5;
Verify:
SELECT RollNo, Name, DeptID, Marks
FROM Student;
Output (Incorrect):

```
mysql> UPDATE Student
    -> SET Marks = Marks + 5;
Query OK, 10 rows affected (0.01 sec)
Rows matched: 10  Changed: 10  Warnings: 0

mysql> SELECT RollNo, Name, DeptID, Marks
    -> FROM Student;
+--------+----------+--------+-------+
| RollNo | Name     | DeptID | Marks |
+--------+----------+--------+-------+
|    101 | Aravind  |      1 |    90 |
|    102 | Bhavya   |      1 |    83 |
|    103 | Chandran |      1 |    95 |
|    104 | Deepak   |      2 |    65 |
|    105 | Esha     |      2 |    60 |
|    106 | Farhan   |      2 |   100 |
|    107 | Gowri    |      3 |    45 |
|    108 | Hari     |      3 |    50 |
|    109 | Indra    |      3 |    73 |
|    110 | Arun     |      2 |    83 |
+--------+----------+--------+-------+
10 rows in set (0.00 sec)
```

Restore the Original Data

Since the incorrect query modifies every row, restore the original marks before demonstrating the correct query:

```
UPDATE Student
SET Marks = CASE RollNo
   WHEN 101 THEN 85
   WHEN 102 THEN 78
   WHEN 103 THEN 90
   WHEN 104 THEN 60
   WHEN 105 THEN 55
   WHEN 106 THEN 95
   WHEN 107 THEN 40
   WHEN 108 THEN 45
   WHEN 109 THEN 68
   WHEN 110 THEN 78
END;
```

Run the corrected query:

```
UPDATE Student
```

SET Marks = Marks + 5
WHERE DeptID = 2;

```
mysql> UPDATE Student
    -> SET Marks = Marks + 5
    -> WHERE DeptID = 2;
Query OK, 4 rows affected (0.02 sec)
Rows matched: 4  Changed: 4  Warnings: 0

mysql> SELECT RollNo, Name, DeptID, Marks
    -> FROM Student;
+--------+----------+--------+-------+
| RollNo | Name     | DeptID | Marks |
+--------+----------+--------+-------+
|    101 | Aravind  |      1 |    90 |
|    102 | Bhavya   |      1 |    83 |
|    103 | Chandran |      1 |    95 |
|    104 | Deepak   |      2 |    70 |
|    105 | Esha     |      2 |    65 |
|    106 | Farhan   |      2 |   105 |
|    107 | Gowri    |      3 |    45 |
|    108 | Hari     |      3 |    50 |
|    109 | Indra    |      3 |    73 |
|    110 | Arun     |      2 |    88 |
+--------+----------+--------+-------+
10 rows in set (0.00 sec)
```

9) Given Query

```
SELECT s.Name, s.Marks,
      (SELECT DeptName FROM Department WHERE DeptID =
s.DeptID) AS DeptName,
      (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS
HeadOfDept
FROM Student s
WHERE s.DeptID IN (
   SELECT DeptID
   FROM Department
   WHERE DeptName IN ('Computer Science', 'Electronics')
);
```

Error:

The query uses multiple subqueries to retrieve DeptName and HOD, and

another subquery in the WHERE clause. This causes unnecessary repeated lookups and reduces query performance.

Correct Query:

```
SELECT s.Name,
       s.Marks,
       d.DeptName,
       d.HOD AS HeadOfDept
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID
WHERE d.DeptName IN ('Computer Science', 'Electronics');
```

Explanation:

The INNER JOIN retrieves the department name and HOD in a single operation. The WHERE clause filters only the required departments, eliminating the need for multiple subqueries.

Optimization:

Replacing subqueries with an INNER JOIN reduces repeated table lookups, improves execution time, and makes the query easier to read and maintain.

MySQL Execution

Run the given query:

```
SELECT s.Name, s.Marks,
       (SELECT DeptName FROM Department WHERE DeptID =
s.DeptID) AS DeptName,
       (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS
HeadOfDept
FROM Student s
WHERE s.DeptID IN (
   SELECT DeptID
   FROM Department
   WHERE DeptName IN ('Computer Science', 'Electronics')
);
```

Incorrect Output:

```
mysql> SELECT s.Name, s.Marks,
    ->        (SELECT DeptName FROM Department WHERE DeptID = s.DeptID) AS Dep
tName,
    ->        (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS HeadOfDe
pt
    -> FROM Student s
    -> WHERE s.DeptID IN (
    ->     SELECT DeptID
    ->     FROM Department
    ->     WHERE DeptName IN ('Computer Science', 'Electronics')
    -> );
+----------+-------+-----------------+------------+
| Name     | Marks | DeptName        | HeadOfDept |
+----------+-------+-----------------+------------+
| Aravind  |    90 | Computer Science | Dr. Rao    |
| Bhavya   |    83 | Computer Science | Dr. Rao    |
| Chandran |    95 | Computer Science | Dr. Rao    |
| Deepak   |    70 | Electronics      | Dr. Iyer   |
| Esha     |    65 | Electronics      | Dr. Iyer   |
| Farhan   |   105 | Electronics      | Dr. Iyer   |
| Arun     |    88 | Electronics      | Dr. Iyer   |
+----------+-------+-----------------+------------+
7 rows in set (0.02 sec)
```

Run the corrected query:

```
SELECT s.Name,
    s.Marks,
    d.DeptName,
    d.HOD AS HeadOfDept
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID
WHERE d.DeptName IN ('Computer Science', 'Electronics');
```

Correct Output:

```
mysql> SELECT s.Name,
    ->        s.Marks,
    ->        d.DeptName,
    ->        d.HOD AS HeadOfDept
    -> FROM Student s
    -> INNER JOIN Department d
    -> ON s.DeptID = d.DeptID
    -> WHERE d.DeptName IN ('Computer Science', 'Electronics');
+----------+-------+------------------+------------+
| Name     | Marks | DeptName         | HeadOfDept |
+----------+-------+------------------+------------+
| Aravind  |    90 | Computer Science | Dr. Rao    |
| Bhavya   |    83 | Computer Science | Dr. Rao    |
| Chandran |    95 | Computer Science | Dr. Rao    |
| Deepak   |    70 | Electronics      | Dr. Iyer   |
| Esha     |    65 | Electronics      | Dr. Iyer   |
| Farhan   |   105 | Electronics      | Dr. Iyer   |
| Arun     |    88 | Electronics      | Dr. Iyer   |
+----------+-------+------------------+------------+
7 rows in set (0.00 sec)
```

10) Given Expression

{ s.Name | Student(s) (t) (Student(t) t.DeptID = 2 ∧ ∀ ∧ → s.Marks > t.Marks) }

Intent: Find students who scored more than **at least one** student in Department 2.

Error:

The expression incorrectly uses the universal quantifier (∀), which means the student's marks must be greater than **every** student in Department 2. However, the requirement is to find students who scored more than **at least one** student in Department 2.

Correct Expression:

{ s.Name | Student(s) ∧ ∃t (Student(t) ∧ t.DeptID = 2 ∧ s.Marks > t.Marks) }

Explanation:

The existential quantifier (∃) checks whether there exists at least one student in Department 2 whose marks are lower than the current student's marks. This matches the intended requirement.

Optimization:

Using the correct quantifier (∃) produces the intended result and avoids unnecessarily strict conditions.

Expected Result

Using the given data, Department 2 students have marks **60, 55, and 95**.
Any student with marks greater than **at least one** of these values (i.e., greater than 55 or 60) is included.

Name
Aravind
Bhavya
Chandran
Deepak
Farhan
Indra